

ETH
Department of Information Technology and Electrical Engineering

VLSI 2: From Netlist to Complete System on Chip
Spring Semester 2026

CROCDIC: A Dual-Module CORDIC Hardware Accelerator with DMA on the Croc SoC

Final Project

Adrian Trummer, atrummer@ethz.ch
Steven Cheng, stcheng@ethz.ch

June 2026

Prof. Dr. Luca Benini
Dr. Frank K. Gürkaynak

1. Introduction

1.1 Motivation and overview

This project aims to accelerate the computation of sine, cosine, arctangent, square root, reciprocal, and inverse square root, using a hardware implementation of the CORDIC (COordinate Rotation DIgital Computer) algorithm on the Croc SoC. Additional DMA capabilities allow the hardware to read in and write back a variable number of elements without CPU involvement. This enables the system to “bulk process” an array of mathematical calculations in hardware and store them in memory directly, bypassing the need for the processor to execute additional instructions for the transactions.

The `crocdic_top` module is integrated into Croc’s user-domain, where the processor can communicate with it using the OBI Subordinate Interface via memory-mapped registers. Additionally, the `crocdic_top` module can issue read and write requests to the SRAM independently using the OBI Manager Interface. Within the `crocdic_top` module, a Finite State Machine (FSM) handles the variable number of reads and writes to memory, as well as the computation of the selected mathematical function using two `crocdic_cordic` modules running in parallel.

The dual-module CORDIC hardware accelerator with DMA results in a significant speedup over software for all mathematical functions implemented. The achieved performance gain ranges from a 39x speedup over software for the computation of 64 $\text{atan}(y/x)$ calculations, up to a 60x speedup over software for the computation of 70 $\text{sqrt}(x)$ calculations. These results demonstrate the immense performance gains achievable with CROCDIC.

1.2 The CORDIC algorithm

The CORDIC algorithm is a mathematical technique used to compute certain functions, including sine and cosine, arctangent, and square root, using only additions, subtractions, and bit-shifts, thus avoiding more difficult and expensive operations to implement in hardware, such as multiplication and division. This is accomplished by representing the target calculation as a sequence of small, fixed-angle rotations of a 2D vector, where each rotation step progressively decreases. After a certain number of steps, the vector converges to the target calculation.

CORDIC depends on a combination of two sets of modes to select the mathematical function to be computed. The first set consists of Rotation mode, which takes a known angle as input and calculates coordinates, and Vectoring mode, which takes known coordinates as input and returns an angle. Rotation mode is used for computing the cosine and sine values of an angle, and Vectoring mode is used for calculating the arctan, square root, and reciprocal of the input value. The second set of modes describes the coordinate system used. Circular mode is used for computing the trigonometric functions sin, cos, and arctan, Linear mode for taking a reciprocal, and Hyperbolic mode for calculating a square root. By setting the correct combination

of modes and providing the inputs in the appropriate format, all of these mathematical functions can be computed using the same CORDIC function, and in CROCDIC’s case, the same `crocdic_cordic` hardware module.

1.3 CORDIC in hardware

To ensure a hardware-friendly implementation, all input/output values are represented as fixed-point numbers in the Q2.14 format, which consists of 1 sign bit, 1 integer bit, and 14 fractional bits. Thus, to convert from the real decimal value to the fixed-point representation, a multiplication by 2^{14} is required, followed by subsequent truncation. Similarly, to convert from the fixed-point representation to the real decimal value, a division by 2^{14} is performed. With 1 sign bit and 1 integer bit, the representable range of this number format is $[-2, 2)$ which is sufficient for representing angles in radians in the first and fourth quadrant of the unit circle where the CORDIC algorithm converges. Additionally, the CORDIC algorithm is designed so that intermediate values stay bounded within that range throughout the iterations. By using the standard two’s complement representation, addition and subtraction work identically in hardware, whether the bits are interpreted as signed or unsigned. Only operations like arithmetic right-shifts and comparisons need to explicitly treat the value as signed, which is handled by appropriate `$signed()` casts in SystemVerilog.

Hardware acceleration is particularly beneficial for this type of workload because, in addition to its hardware-friendly operations like shifts, adds/subtracts, and table lookups, CORDIC is inherently repetitive and can be parallelized, as the calculation of elements in an array is independent from each other. CROCDIC exploits this by implementing two `crocdic_cordic` modules running in parallel, allowing multiple values to be computed simultaneously.

Our hardware CORDIC module and its software-side tests used for functional verification are based on the source-available reference implementation by Nikolaos Kavvadias¹. Test vectors and expected outputs used to validate our hardware implementation were derived from this reference, ensuring bit-accurate agreement between the original fixed-point software model and our hardware accelerator.

¹ N. Kavvadias, *kvcordic: Multi-function, universal, fixed-point CORDIC*. GitHub repository, Version 1.1.0. Available: <https://github.com/nkkav/kvcordic/blob/master/sw/cordic.c>. Accessed: Jun. 28, 2026.

2. Design

2.1 The CROCDIC implementation in the user domain

Figure 2.1 shows the block diagram of the CROCDIC implementation in the user domain of Croc. The `crocdic_top` module is integrated into the user domain of Croc and interacts with the CPU using the OBI Subordinate Interface via memory-mapped registers, shown in Table 2.1. In this way, the CPU configures the addresses of the source and destination arrays, the mathematical operation, and the number of elements to be computed by the CORDIC module. The CPU also begins a computation by writing to a start register and continuously polls a done register until the accelerator has finished processing all elements. Once a computation is initiated, the CORDIC modules access the source and destination arrays directly with DMA, using the OBI Manager Interface. Calculations can then begin for the specified mathematical operation already configured in the register file by the CPU. Lookup tables necessary for the CORDIC algorithm are defined in hardware and referenced within the CORDIC modules for maximum speed.

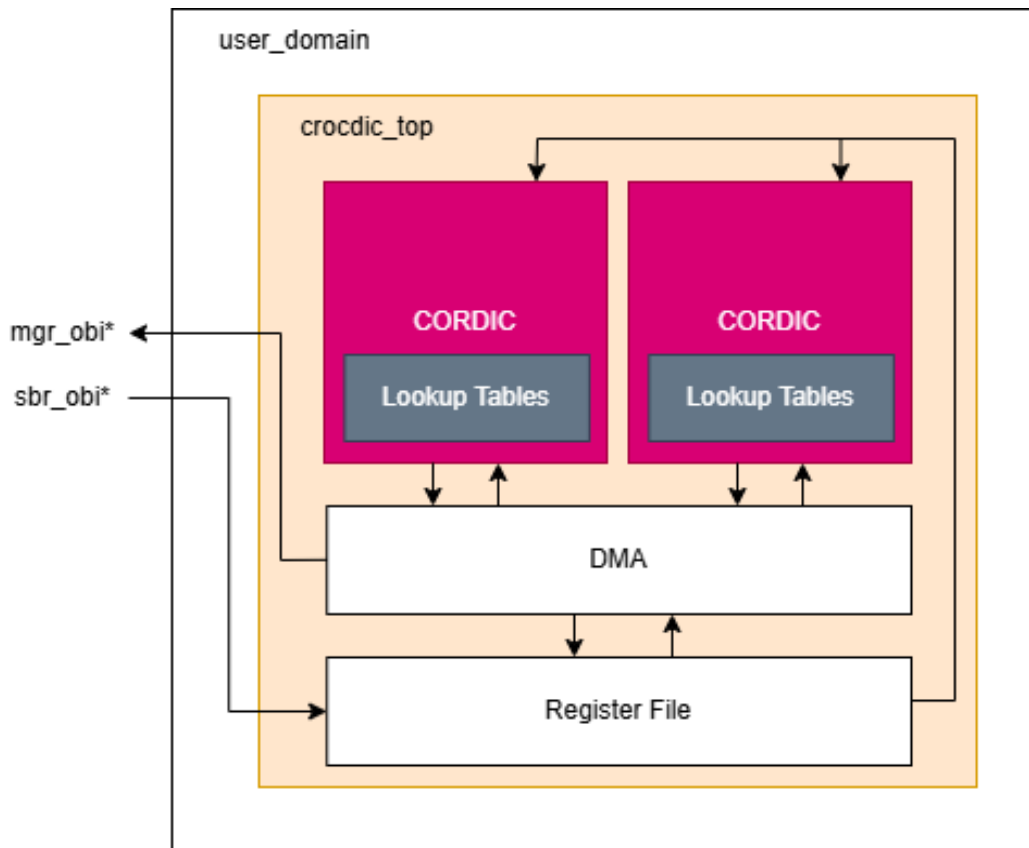


Figure 2.1: Block diagram of CROCDIC implementation in the Croc user domain

Table 2.1: Memory-mapped registers

Register Name	Address	Access	Purpose
STUDENT_NAMES	0x2000_0000	Read	Returns a zero-terminated ASCII string identifying the group
CROCDIC_START	0x2000_0080	Write	Writing any value triggers the accelerator to begin processing
CROCDIC_OPERATION	0x2000_0084	Write	Selects the operation to perform
CROCDIC_SOURCE_ARRAY_ADDRESS	0x2000_0088	Write	Base address in SRAM of the input array to be processed
CROCDIC_NR_OF_ELEMENTS	0x2000_008C	Write	Number of elements in the source array to process
CROCDIC_DESTINATION_ARRAY_ADDRESS	0x2000_0090	Write	Base address in SRAM where computed results are written
CROCDIC_DONE	0x2000_0094	Read	Polled by software; returns a non-zero value once processing has completed

2.2 FSM and data flow in the CROCDIC top module

The left flow chart of Figure 2.2 shows the operation of the `crocdic_top` module. This module is organized into 8 FSM states that manage the CORDIC computation process by reacting to requests from the CPU, issuing read and write requests to the SRAM banks, and starting the 2 `crocdic_cordic` modules.

The `IDLE` state is the default state of the `crocdic_top` module in which it is initialized, and to which it returns once the CORDIC computation process has finished. While in the `IDLE` state, the CPU can interact with the hardware accelerator by reading and writing to the memory-mapped register map of `crocdic_top`. Before starting the computation process, the desired mathematical operation, source array address, number of elements in the source array to be processed, as well as destination array address, are configured by the CPU by writing the appropriate values to the respective addresses of the memory-mapped register file. Subsequently,

the CORDIC computation process can be started by the CPU by issuing a write request to the CROCDIC_START address. Additionally, reading from the STUDENT_NAMES address returns a zero-terminated string containing the names of the students in the group.

The READ_ELEMENTS state is entered once the CPU has issued the start request, and reads in the elements by issuing read requests directly to the SRAM banks via the OBI Manager Interface. Since 32 bits can be read from the SRAM banks using a single read request, two 16-bit elements (in Q2.14 format) can be obtained at a time, motivating our dual-module CORDIC architecture, which processes two 16-bit input values simultaneously. Special care needs to be taken for an odd number of elements, in which case the byte-enable flag is set to only read 16 bits for the last element. Once the read request has been granted, the FSM transitions to the WAIT_READ_RESPONSE state.

In the WAIT_READ_RESPONSE state, the FSM waits for the valid signal to be asserted, signaling that the read request has been completed. After that, it can store the 32-bit response data in two registers, which form the inputs to the two crocdic_cordic modules. Special care needs to be taken for the ATAN operation, as it requires two 16-bit input values in the form of consecutive (x,y) pairs in the source array per single atan(y/x) calculation. Therefore, a second read request is issued for the ATAN case to read in the second (x,y) input pair for the second CORDIC module. In other words, each read request loads the inputs for one CORDIC module during an ATAN calculation. Once the input elements for the two crocdic_cordic modules have been loaded, the FSM transitions to the START_CORDIC state.

In the START_CORDIC state, the start-signals of the two crocdic_cordic modules are asserted for one clock cycle, before transitioning to the WAIT_CORDIC state.

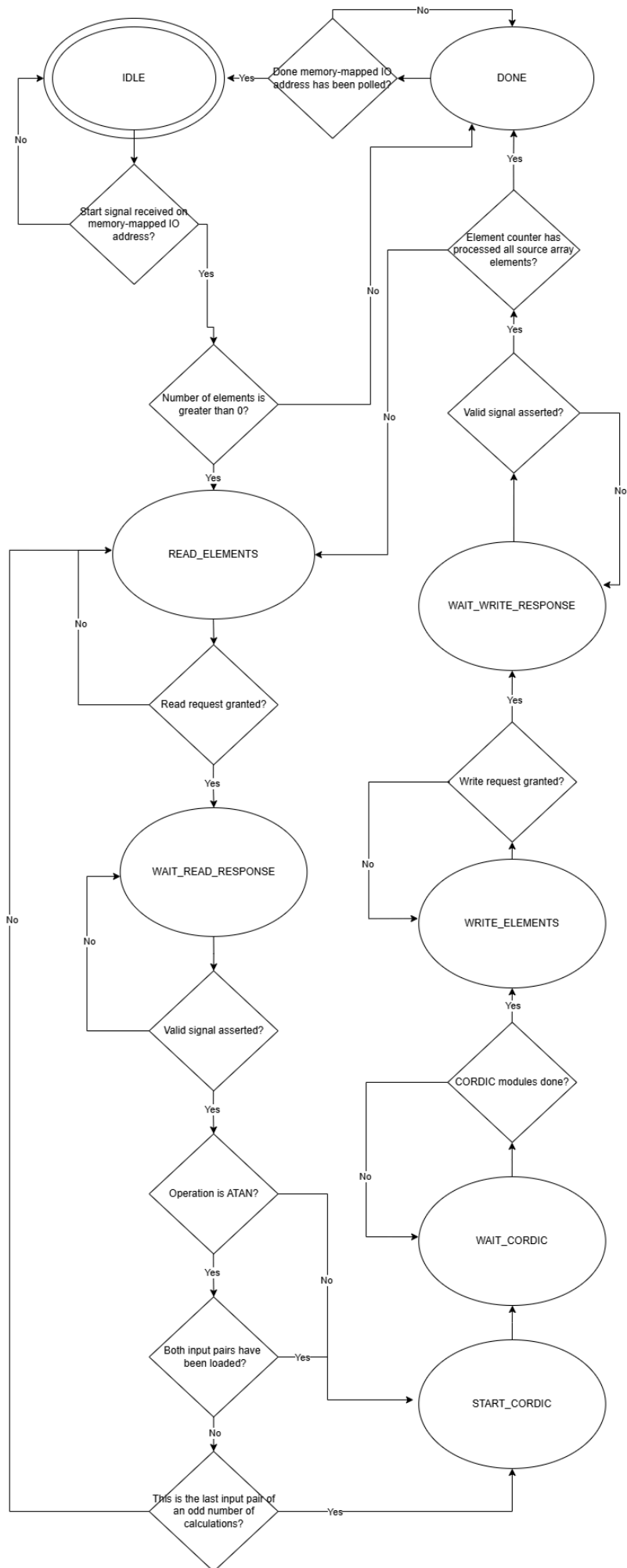
The WAIT_CORDIC state waits until the two CORDIC modules assert their done-signals, after which the two outputs are stored in separate registers, and the FSM transitions to the WRITE_ELEMENTS state.

In the WRITE_ELEMENTS state, the two 16-bit outputs produced by the two crocdic_cordic modules are written back to the appropriate positions in the destination array using a single 32-bit write request. Again, special care needs to be taken for an odd number of elements, such that the byte-enable flag is set to only write back 16 bits for the last element. Once the write request has been granted, the FSM transitions to the WAIT_WRITE_RESPONSE state.

In the WAIT_WRITE_RESPONSE state, the FSM waits for the valid signal to be asserted, signaling that the write request has been completed. After that, it can update the element counter by incrementing it by the number of source array elements it has just processed. After comparing the element counter to the total number of elements to be processed as configured by the CPU in the register file, it either transitions back to the READ_ELEMENTS state to process the next batch of input elements or transitions to the DONE state once all source array elements have been processed.

In the `DONE` state, all CORDIC calculations have finished, and the FSM waits for the CPU to poll the `CROCDIC_DONE` address. Once the CPU issues a read request to that address, it returns a non-zero response value to the CPU signaling that the computation has finished, and the FSM returns to the `IDLE` state, where it can wait for new requests from the CPU.

FSM of crocdic_top



FSM of crocdic_cordic

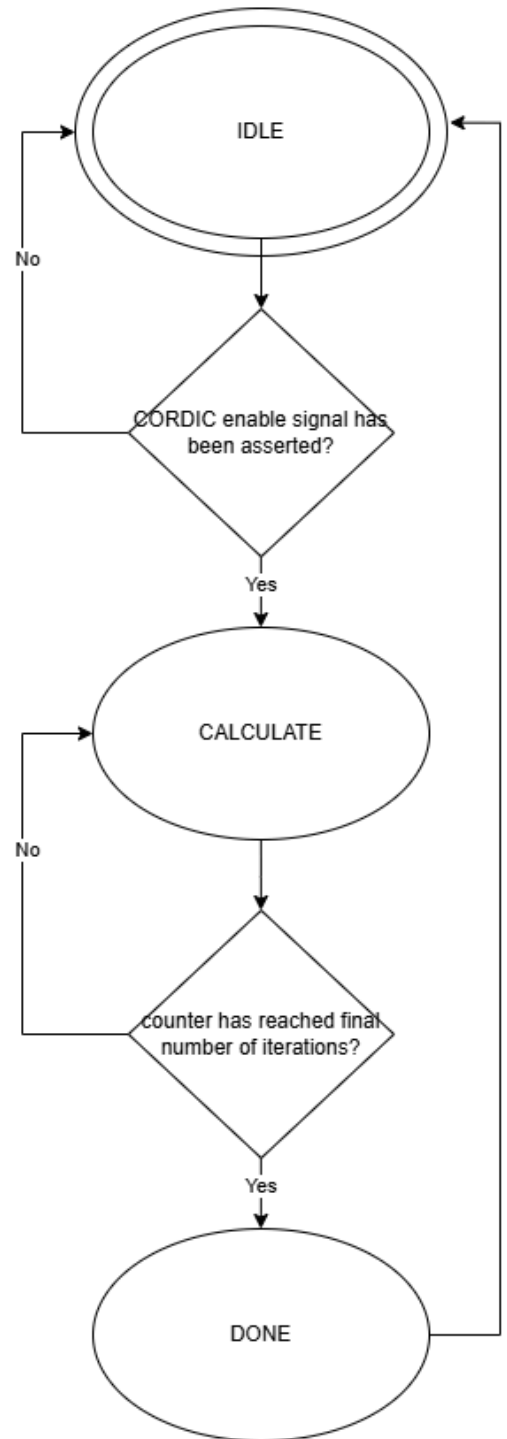


Figure 2.2: Flow charts of the top module FSM (left) and the CORDIC module FSM (right)

2.3 FSM and calculation in the CORDIC module

The `crocdic_cordic` module contains a hard-wired look-up table needed for the CORDIC calculations and a simple FSM with 3 states, responsible for performing one CORDIC calculation at a time by iterating through a fixed number of iterations. The operation of the FSM is shown in the right flow chart of Figure 2.2.

The default state of the `crocdic_cordic` module is the `IDLE` state, where the module resides outside of running CORDIC calculations. Depending on the `operation_i` input signal, the starting conditions of the CORDIC calculation are configured. Once the input enable signal is asserted, the FSM transitions to the `CALCULATE` state.

In the `CALCULATE` state, the module calculates each CORDIC iteration of the x , y , and z values combinationaly and assigns them to the next state of their respective registers. Due to the use of two's complement representation, addition and subtraction work identically in hardware for both signed and unsigned signal interpretation. For operations where the sign explicitly matters, such as arithmetic right shifts and comparisons, `$signed()` casts are used to handle those operations correctly. The counter is incremented each iteration, and the FSM stays in the `CALCULATE` state until the counter has reached the final number of iterations for the selected mathematical operation. After all iterations have completed, the FSM switches to the `DONE` state.

In the `DONE` state, the `crocdic_cordic` module outputs the final output value of the calculation via the 16-bit `output_element_o` signal, as well as a `done` signal asserted for one clock cycle. The module then transitions back to the `IDLE` state, where it resides until the enable input signal is asserted again.

2.4 Design Tradeoffs

The `crocdic_cordic` module was implemented to complete one calculation in 14 (respectively 15 for square root) iterations, where the values of each iteration are computed using combinatorial logic in one clock cycle. This does not lead to any timing violations, as the combinatorial path for computing the values of one iteration still fits within the target clock frequency of 100 MHz. One alternative design decision would be to perform partial loop unrolling, where multiple CORDIC iterations are computed via combinatorial logic in one clock cycle. Due to time constraints, we were unable to check whether the resulting combinatorial path would still fit for our desired clock frequency of 100 MHz. If possible, this would be a straightforward way to significantly improve the performance of the hardware accelerator, as each CORDIC calculation would require fewer clock cycles to complete.

While the Q2.14 number format used for the CORDIC input and output values is fixed by our reference C implementation, there are tradeoffs associated with the selection of number representation. By using 16 bits in total per element, CROCDIC is able to read and write 2 elements at a time using the 32-bit wide OBI bus, which is advantageous for the throughput of our system. By having 1 sign bit and 1 integer bit, a dynamic range of $[-2, 2)$ is achieved, which

is sufficient for representing angles in radians in the first and fourth quadrants of the unit circle. For angles outside that range, additional pre-processing is required to map them to the allowed input range. For calculations such as square root and reciprocal, these range limitations mean that only small values can be computed with our CORDIC implementation, and additional pre-processing would be required to handle larger input values. By allocating 14 bits to the fractional part, a resolution limit (LSB step size) of 2^{-14} is achieved, which yields roughly 4 decimal places of reliable resolution. The 14 fractional bits also correspond to the total number of iterations inside our CORDIC module, as every iteration resolves exactly one bit of fractional accuracy. This means that running more than 14 iterations inside our CORDIC module is pointless, as our number format only supports 14 fractional bits, and thus finer-resolution values would be truncated down to zero. Thus, making use of a number format with more fractional bits could increase the accuracy of our results, but at the expense of a higher area consumption due to the larger number format, longer calculation times as more read and write requests would be needed to fetch and write back the larger elements to memory, as well as a higher number of iterations in the CORDIC module due to more iterations to compute the finer-precision value.

Another major design tradeoff is the number of CORDIC modules used in the hardware accelerator. The proposed design employs two CORDIC modules running in parallel, as this fits well with the number of 16-bit elements to be read in and written back at a time using the 32-bit wide OBI bus. However, the performance of the system could be further improved by instantiating even more CORDIC modules in parallel, which was not implemented due to time constraints. In a future implementation of CROCDIC, the number of CORDIC modules used would depend on the desired degree of acceleration and the tradeoff between performance and area.

3. Implementation

The implementation followed the reference flow using Yosys, OpenROAD, and KLayout. Since no modifications were made to the main SoC configurations outside of the user domain, such as changing the number of SRAM banks or the number of words inside each SRAM bank, there were not many required modifications to the reference backend flow.

One modification required to meet the physical requirements was to change the chip size, including the searing, to 2500 um (wide) x 2000 um (high), as the reference was set to 2000 um x 2000 um. To modify this, the `chipW` TCL variable in the `01_floorplan.tcl` script was increased by 500 from 1916 to 2416.

Furthermore, the pinouts of the chip were changed since the reference configuration did not match the specifications. For this, the `padring.tcl` script was modified to match the pinout specified in the provided `bondpad_centroids.csv` file, ensuring that 18 pads are placed on each of the 2 long horizontal edges with equal spacing between them, and 14 pads are placed on each of the 2 short vertical edges in an equidistant manner.

After making these modifications to the OpenROAD flow, it was able to run according to specifications. The resulting OpenROAD output was further processed using KLayout by running the scripts `./def2gds-croc` and `./run_drc-croc` to generate the GDSII file from the OpenROAD DEF output file. DRC was then performed on the resulting layout.

The DRC was performed each time changes to the RTL were made, and the backend flow was rerun. A common pattern reported was that our design exhibited the expected `metall_pin_Offgrid` violations in the SRAM macro, as well as `M1.b` violations in the `croc_chip`, which are minimum spacing violations on the Metal 1 layer. After resolving these `M1.b` violations manually a few times in KLayout, a programmatic fix was added for these violations by inspecting which cells were causing these issues, which were `DFFRQX3` and `AND4X2`, and adding the `set_placement_padding` command to the `02_placement.tcl` script in the OpenROAD flow before the global placement. In this way, an additional reserve spacing on both sides of the problematic cells was provided, which resulted in a clean DRC after rerunning the OpenROAD flow.

After achieving a DRC-clean layout, a successful LVS ensured the layout matches the original schematic and is ready for manufacturing.

4. Testing

4.1 Test Programs

The performance of CROCDIC was tested and compared with a reference software CORDIC implementation, using five C programs for each of the CORDIC operation capabilities, and an additional program that combines two operations. These capabilities are sine, cosine, arctangent, square root, and reciprocal. The additional program calculates a reciprocal square root by first calculating a square root and then the reciprocal of the output. The CORDIC algorithm is capable of calculating other functions, but testing is limited to these 6 calculations for consistency with the reference software implementation. The different CORDIC calculation tests are each implemented in their own program for modularity and due to limitations in memory.

In each of the programs, the source array of values to be computed by the software and hardware CORDIC implementations is defined in Q2.14 format, and a destination array of the appropriate size is declared for each implementation. Both the source and destination arrays expect values in the Q2.14 format for consistency and to avoid memory overflow that may result during conversion to readable decimal values. The reference software CORDIC function is defined outside of the main function, inspired by a source-available reference C implementation, discussed in Section 1.3. This reference function is not defined in its own reusable file in order to prevent memory overflow. In the main function, the reference software function is called in a for-loop using elements from the source array as inputs, and the calculated outputs are assigned as elements in the destination array. Afterwards, the hardware implementation of the CORDIC algorithm is initiated by assigning the calculation arrays and parameters to their corresponding addresses and then writing to the CROCDIC_START address, all through memory-mapped I/O (MMIO). The program then waits until CROCDIC finishes all calculations before proceeding to ensure accurate timing. Three timestamps are declared using `get_mcycle()` within the main function and used to measure and compare the timing of the software and hardware implementations. The timestamps are taken directly before the software calculations initiate, once it finishes and before the hardware calculations initiate, and once the hardware calculations complete.

After the calculations are complete, the elements of the destination array for the software implementation and the hardware implementation are compared in a for-loop, and a mismatch notification is printed for each element mismatch. At the end of the tests, the number of cycles taken for the software and hardware implementations is reported along with the total number of mismatches.

4.2 Edge Cases

Two more programs were also added to test the case of an odd number of calculations, in which only one of the dual CORDIC modules would produce an output on the final iteration of calculations. In these tests, the source array size is expected to result in one less calculation, but

the destination array size is kept the same as in the even calculation tests and has its elements all set to 9, in order to determine whether the CORDIC module properly leaves the final element untouched. An additional comparison is implemented in the odd-calculations tests to check this.

Inputs falling outside of the input range of the CORDIC algorithm would require pre-processing, particularly range reduction to provide an input within the input range that would result in the same calculation output. For example, the algorithm accepts angles between $-\pi/2$ and $\pi/2$ for sine, so an input angle of $3\pi/4$ would require an angle reduction to $\pi/4$ for an equivalent sine calculation. Due to the memory limitations and because all tests would produce correct results with pre-processed inputs, range reduction was beyond the scope of CROCDIC and not implemented.

4.3 Simulations

The RTL simulation was performed by Verilator iteratively during the RTL design stage, and the post-layout simulation was performed by QuestaSim after DRC and LVS to ensure the design still functioned as intended after the backend flow. A few changes from the reference flow were necessary for the post-layout simulation to run properly and should be noted for future runs. The `ifdef` block labeled `TARGET_NETLIST_OPENROAD` was copied into the testbench from the Exercise 10 directory, with the corresponding `define` statement placed in the `vsim/compile_netlist.tcl` file. The `croc_chip.sv` RTL file was also replaced by the OpenRoad output `croc.v` file in the `compile_netlist.tcl` file such that QuestaSim targets the post-layout design, and unnecessary blocks generated by Bender that only contained include-directories and caused issues were deleted. Additionally, the `compile_tech.tcl` file was modified to only include the technology files available in our flow. The `vsim/work` directory then needed to be deleted and rebuilt to ensure the previously present `croc_chip.sv` RTL file would not be reused as a target. The `ihp13/empty_cells.v` library was also copied over from the `ihp13` directory in Exercise 10 to resolve an error regarding undefined filler modules.

The final version of the RTL and the post-layout design both successfully pass all simulation tests with no mismatches between software and hardware results. A comparison of the number of cycles taken is shown in the Results section.

5. Results

After the backend flow, CROCDIC passed DRC and LVS. Figure 5.1 shows the module view of the final CROCDIC layout in OpenRoad, with the user domain in purple, which consists primarily of the `i_crocdic_top` instance with two `crocdic_cordic` instances and ROM with the designer names. The user domain takes an area of $72,181.469 \mu\text{m}^2$, or 10 kGE, and contains a total of 7649 leaf instances. For comparison, the `i_core_wrap` instance takes an area of $145,331.021 \mu\text{m}^2$, shown in Figure 5.1 as the blue region. Figure 5.2 shows the area breakdown of instances in CROCDIC. The core utilization is 27.54%.



Figure 5.1: Module view of final CROCDIC layout in OpenRoad

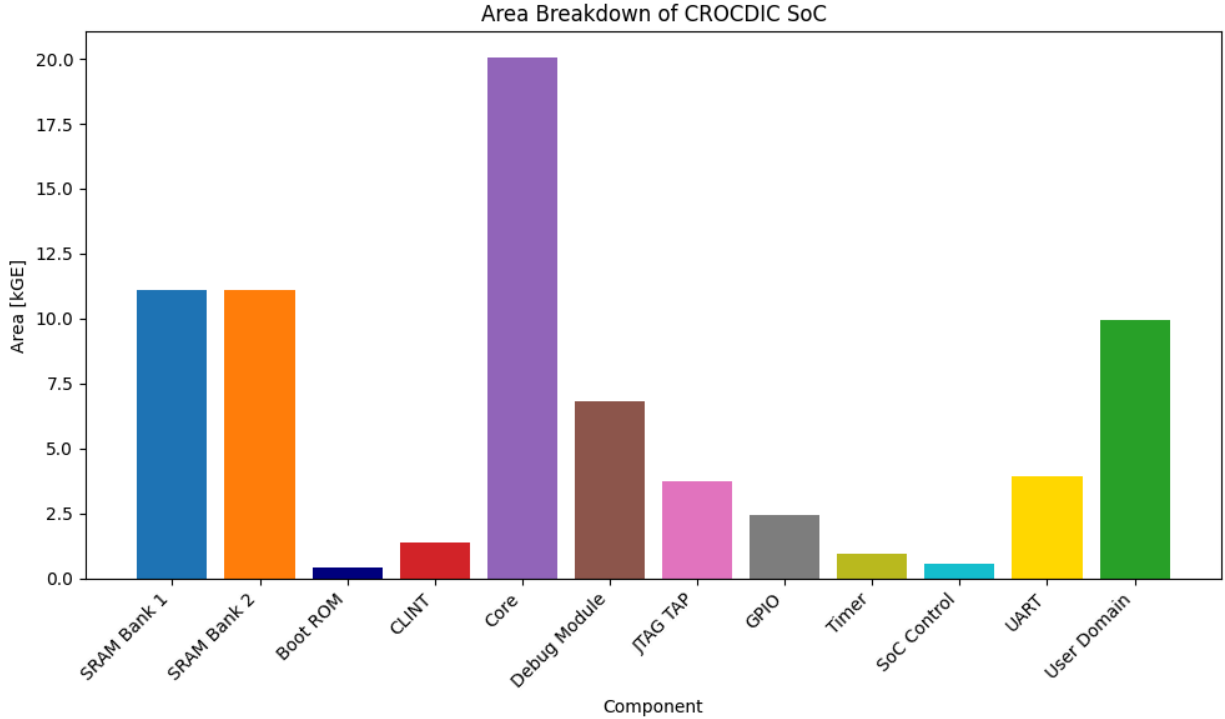


Figure 5.2: Area breakdown of instances in the CROCDIC SoC

The final CROCDIC design after the finishing step of the backend had no timing violations. At a system clock frequency of 100 MHz, the worst slack for the setup time is 1.08 ns, while the worst slack for the hold time is zero. The user domain is not among the paths that result in the worst slacks.

Table 5.1 shows a comparison of the number of clock cycles taken by the software and hardware implementations for each operation tested. The speed results are the same between RTL simulation and post-layout simulation as expected, because the calculations take the same number of cycles, and there were no timing violations in the final placed-and-routed design. As shown, the hardware implementation results in a significant speedup of up to 60 times in the case of the square root operation. The main sources of speedup include the parallel CORDIC units and the DMA-based array access. In future implementations, additional CORDIC units can be added for increased parallel processing with the tradeoff of area. As shown in Figure 5.1, if the goal is to specialize in CORDIC calculations, there is plenty of remaining area for additional CORDIC units.

Table 5.2 shows a statistical power analysis and stimulus-based power analysis with stimulus generated from an array of sine calculations using the corresponding test program. The total power is similar between the two analyses. Table 5.3 shows IR drop analyses for VDD and VSS.

Table 5.1: Speed comparison for software and hardware CORDIC implementations

Operation	SW Calculation Time (Cycles)	HW Calculation Time (Cycles)	Speedup (SW Time/HW Time)
Sine	27,559	622	44.31
Cosine	27,559	623	44.24
Arctangent	35,650	917	38.88
Square root	53,960	897	60.16
Reciprocal	51,997	1,174	44.29
Inverse square root	81,754	1,559	52.44

Table 5.2: Statistical and stimuli-based power analyses at 1.2 V supply voltage and 100 MHz system clock frequency.

	Internal Power (mW)	Switching Power (mW)	Leakage Power (mW)	Total Power (mW)
Statistical Stimulus	11.9	7.2	0.0042	19.1
	62.3%	37.7%	0.0%	100.0%
Sine Calculation Stimulus	14.8	7.2	0.0043	21.9
	67.2%	32.8%	0.0%	100.0%

Table 5.3: Statistical VDD and VSS IR drop analyses at corner TT

	VDD (supply droop)	VSS (ground bounce)
Total Power (mW)	20	20
Supply Voltage (V)	1.2	0
Worst-case Voltage (V)	1.19	0.007
Average IR Drop (mV)	5.11	4.57
Worstcase IR Drop (mV)	7.54	7
Percentage Drop	0.63%	0.58%